TOTALISATION

L'origine de l'art de l'excellence pour la réalisation dans le domaine du logiciel

Une déclaration de vision - Un manifeste pour l'excellence totale

PARTIE A - L'ORIGINE

Léo Michel Bélanger

Publié par ORIGINALIUM | TOTALISATION

© Léo Michel Bélanger - Tous droits réservés
Images et couverture par Soléane Bélanger
Aucune reproduction, même partielle, sans autorisation écrite de l'auteur.

Ce livre est un manifeste philosophique et une declaration de vision.
Il ne contient ni exercices, ni programmes hebdomadaires, ni guides pratiques.
C'est une invitation a contempler, a comprendre, et a incarner
une vision totale de l'excellence dans le domaine du logiciel.

© Léo Michel Bélanger - Tous droits réservés

Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM | ṮṰĻṢṬ~

Montreal, Quebec, Canada

Note sur la marque ṮṰĻṢṬ~

Le nom **ṮṰĻṢṬ~** est la transcription symbolique du mot **TOTALISATION**. Chaque caractère phonétique représente une syllabe : Ṯ pour *TO*, Ṱ pour *TA*, Ļ pour *LI*, Ṣ pour *SA*, Ṭ pour *TI*, et ~ pour *ON*. Cet encodage visuel n'est pas un artifice graphique - c'est une declaration : la totalisation ne se lit pas, elle se vit, syllabe par syllabe, dans chaque acte de creation logicielle.

Table des matieres

Partie A - L'Origine

Préambule - La naissance d'une vision p. 4

Chapitre 1 - Qu'est-ce que ṮṰĻṢṬ~ (Totalisation) ? p. 6

Chapitre 2 - L'état du monde du logiciel : un diagnostic honnête p. 9

Chapitre 3 - L'art versus la technique dans le logiciel p. 12

Chapitre 4 - Les origines philosophiques de l'excellence p. 16

Note de fin de Partie A - Apercu de la Partie B p. 20

PRÉAMBULE

La naissance d'une vision

Pourquoi l'excellence dans le logiciel est-elle un art, et non une simple technique ? Pourquoi, malgré des décennies de méthodologies, de frameworks, d'outils et de certifications, le génie logiciel demeure-t-il si profondément marqué par l'échec silencieux ? Ce préambule ne répond pas a ces questions. Il pose les fondations pour que tu puisses les entendre autrement.

I

l existe, dans le monde du logiciel, une crise dont on ne parle jamais assez franchement. Ce n'est pas une crise de technologie. Ce n'est pas non plus une crise de talent. C'est une crise de vision - une incapacité collective a voir le logiciel pour ce qu'il est vraiment : un acte de creation totale, un acte ou l'avant, le pendant et l'apres ne sont pas trois moments distincts, mais une seule et meme realite en mouvement permanent.

Tu as sans doute deja vecu ce moment particulier. Tu livres une fonctionnalite. Elle fonctionne. Les tests passent. Le client signe. Et pourtant, quelque chose en toi sait que ce n'est pas tout a fait juste. Pas completement mauvais, mais pas completement bon non plus. Il manque quelque chose d'essentiel, quelque chose que les metriques ne capturent pas et que les retrospectives n'identifient jamais clairement. Ce quelque chose, c'est ce que ce livre appelle l'excellence totale - et

c'est precisement ce que la philosophie de ŢŢĻṢŢ~ (Totalisation) cherche a nommer, a incarner, et a transmettre.

Ce livre n'est pas un manuel. Ce n'est pas un guide de bonnes pratiques. Ce n'est pas une liste de choses a faire ou a eviter. C'est un manifeste philosophique. C'est une declaration. C'est une invitation a changer profondement ta facon de voir la creation logicielle, non pas en te donnant de nouvelles techniques, mais en te donnant une nouvelle maniere d'habiter ce que tu fais deja.

La crise silencieuse du génie logiciel moderne

Chaque année, des milliards de dollars sont investis dans des projets logiciels qui n'atteignent jamais leur plein potentiel. Ce n'est pas la nouveaute de cette affirmation qui devrait te frapper - c'est sa persistance. Depuis les annees 1970, depuis les premieres etudes sur les projets informatiques en echec, le taux de succes n'a pas fondamentalement progresse. Les methodologies ont change : on est passe de Waterfall a Agile, de RUP a Scrum, de Scrum a SAFe, de SAFe a diverses formes d'organisation fluide. Les outils se sont multiplies de maniere exponentielle. Les langages de programmation se sont sophistiques. Et pourtant, quelque chose resist.

Ce qui resiste, c'est l'absence d'une philosophie fondatrice. On a construit des methodes sur du vide. On a empile des pratiques sur des pratiques sans jamais se demander ce que l'excellence logicielle signifie vraiment, dans sa nature la plus profonde. On a mesure la vitesse, la couverture de tests, le nombre de bugs, le temps de deploiement - et on a confondu ces mesures avec l'excellence elle-meme. C'est comme mesurer la qualite d'une oeuvre musicale uniquement par la precision du tempo, en ignorant completement l'emotion qu'elle transmet.

> *« La crise du logiciel n'est pas une crise d'outils. C'est une crise de sens. Et le*

> *sens ne s'installe pas dans un backlog - il s'incarne dans une vision. »*
>
> - Léo Michel Bélanger, ORIGINALIUM

La Totalisation - ŢŢĻŞŢ~ - est nee de cette conviction : que le logiciel merite une philosophie a sa hauteur. Une philosophie qui ne renie pas la rigueur technique, mais qui la transcende. Une philosophie qui reconnaisse que creer un logiciel, c'est un acte profondement humain, intimement lie a notre capacite a synthétiser, a integrer, a realiser totalement - avant, pendant et apres, en continu, sans interruption.

La Partie A de ce livre, que tu tiens presentement entre tes mains - ou que tu lis sur ton ecran -, te propose un voyage vers l'origine. L'origine de la vision. L'origine du concept de Totalisation. L'origine de la question de l'excellence dans le logiciel. Avant de comprendre ou aller, il faut comprendre d'ou tout cela vient. Et cette exploration de l'origine est elle-meme un acte de Totalisation.

CHAPITRE 1

Qu'est-ce que ŢŢĻŞŢ~ (Totalisation) ?

Avant de parler de methode, avant de parler d'excellence, avant meme de parler de logiciel, il faut nommer ce qui est au coeur de tout ce livre : la Totalisation. Ce mot n'est pas un slogan. C'est un concept vivant, a deux dimensions, qui change profondement la maniere dont on percoit la creation logicielle.

L

a Totalisation est un mot que tu connais deja, au sens ordinaire du terme. Il appartient au lexique courant de la langue francaise. Mais comme beaucoup de mots courants, il a ete use par l'usage quotidien jusqu'a perdre sa charge semantique profonde. Dans le cadre de la philosophie ORIGINALIUM, la Totalisation retrouve toute sa puissance - et meme plus, parce qu'elle acquiert une seconde dimension qui n'existait pas dans son usage habituel.

Dimension 1 - Le Substantif linguistique

Dans son sens premier, celui que tu trouves dans les grands dictionnaires de la langue francaise, la Totalisation designe l'*action d'additionner ; l'action de considerer l'ensemble, le total, de synthetiser.* C'est un geste intellectuel fondamental : celui de rassembler ce qui est disperse, de voir l'ensemble la ou d'autres ne voient que des parties, d'operer une synthese la ou d'autres s'arretent a l'analyse.

Ce geste de synthese est plus rare qu'il n'y parait. Dans le monde du logiciel, on est tres fort pour analyser. On decompose un probleme, on cree des user stories, on segmente en sprints, on decoupes en microservices, on fragmente en composants. L'analyse - la decomposition - est le geste dominant du genie logiciel moderne. Mais la synthese ? La capacite de voir comment tout cela se tient ensemble, comment les parties interagissent pour former un tout coherent, comment le logiciel vit dans son contexte total ? C'est beaucoup plus rare. Et c'est precisement ce que la Totalisation reintroduit comme geste fondateur.

Dimension 1 - Substantif linguistique

"Action d'additionner ; action de considerer l'ensemble, le total, de synthetiser." - La Totalisation dans son sens lexical fondamental est un acte de rassemblement intellectuel. Elle invite a voir l'addition la ou d'autres ne voient que la soustraction, la fragmentation, la decomposition.

Dimension 2 - La Philosophie ORIGINALIUM

Mais la Totalisation, dans la philosophie ORIGINALIUM, ne s'arrete pas au geste de synthese intellectuelle. Elle va beaucoup plus loin. Elle designe la capacite - et l'engagement - de *rassembler et realiser totalement dans son ensemble - avant, pendant et apres - en continu, sans interruption.*

Derriere chacun de ces mots se cache une vision du monde radicalement differente de celle qui domine actuellement le genie logiciel. Decortiquons-la ensemble, parce qu'elle merite qu'on s'y attarde.

Rassembler et realiser totalement : Il ne s'agit pas de faire les choses partiellement ou de les deleguer a un processus. Il s'agit d'une presence totale dans l'acte de creation. Chaque ligne de code, chaque decision d'architecture, chaque conversation avec un utilisateur sont des actes de rassemblement. Tu rassembles ta connaissance technique, ta comprehension du probleme, ton intuition d'artisan, ta rigueur scientifique - et tu realises a partir de cet ensemble unifie.

Dans son ensemble : Le logiciel n'est pas une serie de taches dans un backlog. C'est un systeme vivant, un organisme complexe qui existe dans un contexte humain, organisationnel, technique et temporel. La Totalisation te demande de voir cet ensemble, de le tenir dans ton regard, meme quand tu travailles sur un detail infime.

Avant, pendant et apres : C'est la dimension la plus radicale de la philosophie ORIGINALIUM. L'excellence ne commence pas quand le sprint demarre. Elle ne se termine pas quand le code est en production. Elle existait avant - dans la vision, dans la comprehension du probleme, dans la qualite de la pensee preparatoire. Elle existe pendant - dans chaque geste de creation, dans chaque decision de design. Et elle continue apres - dans la maniere dont le logiciel evolue, dans le soin

apporte a sa maintenance, dans l'attention portee aux utilisateurs qui vivent avec lui.

Dimension 2 - Philosophie ORIGINALIUM

"Rassembler et realiser totalement dans son ensemble - avant, pendant et apres - en continu, sans interruption." - La Totalisation dans sa dimension philosophique est un engagement envers une excellence sans rupture, une presence complete dans l'acte de creation logicielle, de la premiere intuition a la derniere evolution du systeme.

En continu, sans interruption : Ce n'est pas une pratique qu'on applique quand on a le temps. Ce n'est pas une valeur qu'on affiche sur un mur et qu'on oublie dans le feu de l'action. C'est un etat d'etre, une maniere d'habiter son travail en permanence. La Totalisation est continue parce que l'excellence est continue - ou elle n'est pas.

L'étymologie et le sens profond

Le mot « totalisation » vient du latin *totus*, qui signifie « tout entier », « complet », « integral ». Cette racine est fascinante parce qu'elle ne designe pas simplement la somme des parties, mais l'integralite d'une chose consideree comme un tout indivisible. En latin classique, *totus* s'opposait a *omnis* - qui designe « tous les elements d'un ensemble » - en ce qu'il soulignait l'unite, la cohesion, l'intégrite de ce tout.

C'est cette nuance que la philosophie ORIGINALIUM cherche a recuperer et a amplifier dans le contexte du logiciel. Quand on parle de totalisation logicielle, on ne parle pas de tout faire, de tout couvrir, de tout documenter dans une accumulation exhaustive. On parle d'integrite. On parle de cohesion. On parle

d'un logiciel qui est *totus* - entier en lui-meme, coherent dans toutes ses dimensions, unifie par une vision qui traverse chaque couche de sa realisation.

Comment la Totalisation voit le logiciel différemment

La vision dominante dans le monde du logiciel aujourd'hui est une vision fragmentee. On parle de « stacks technologiques », de « silos organisationnels », de « dette technique », d'« impediments ». Le vocabulaire lui-meme trahit une perception eclatee de la realite. Le logiciel est pense comme une serie de problemes a resoudre, de taches a accomplir, d'obstacles a surmonter.

La Totalisation propose une vision radicalement differente. Elle voit le logiciel comme un acte de creation total, comparable dans sa nature - sinon dans ses outils - a d'autres grandes formes d'excellence humaine. Un batisseur de cathédrale medievale ne pensait pas en termes de « stories » a completer. Il avait une vision - la lumiere qui devait traverser la rosace a midi en automne, la hauteur des voutes qui devait inspirer l'elevation de l'esprit, la solidite des fondations qui devait garantir que l'oeuvre traverserait les siecles. Et chaque pierre posee par chaque tailleur de pierre etait un acte conscient au service de cette vision totale.

Le logiciel merite la meme approche. Chaque fonction que tu codes, chaque test que tu ecris, chaque conversation que tu as avec un utilisateur est une pierre dans la cathedrale. La Totalisation te demande de ne jamais perdre de vue la cathedrale - meme quand tu tailles une pierre.

La réconciliation comme acte fondateur

L'une des contributions les plus importantes de la philosophie ORIGINALIUM est l'idee que la Totalisation est fondamentalement un acte de reconciliation. Reconciliation entre la vitesse et la qualite. Reconciliation entre l'agilite et la

rigueur. Reconciliation entre l'innovation et la stabilite. Reconciliation entre l'intuition de l'artisan et la methode du scientifique.

Pendant trop longtemps, ces dimensions ont ete presentees comme des oppositions irreconciliables. Il faudrait choisir entre aller vite et aller bien. Il faudrait choisir entre etre agile et etre rigoureux. Ce faux choix a fait enormement de mal au genie logiciel. Il a produit des equipes qui courent sans savoir ou elles vont, et des equipes qui savent ou elles vont mais n'arrivent jamais a temps.

La Totalisation refuse ce faux dilemme. Elle dit : la vitesse veritable vient de la qualite profonde. Elle dit : l'agilite reelle emerge de la rigueur interieure. Elle dit : l'innovation durable repose sur la stabilite fondamentale. Et elle offre un cadre - le cadre BEASTEST - pour incarner cette reconciliation dans la pratique quotidienne.

CHAPITRE 2

L'état du monde du logiciel : un diagnostic honnête

Il faut avoir le courage de regarder le monde du logiciel tel qu'il est, et non tel qu'on voudrait qu'il soit. Ce diagnostic n'est pas un acte de desespoir - c'est le premier pas vers la guerison. On ne peut soigner que ce qu'on accepte de voir.

S

i tu travailles dans le domaine du logiciel depuis quelques annees - que tu sois developpeur, architecte, gestionnaire de projet, testeur, analyste ou directeur technique -, tu as sans doute une relation ambivalente avec ce que tu vois autour de toi. Tu vois des equipes brillantes. Tu vois de la passion. Tu vois de la creativite. Et en meme temps, tu vois des projets qui n'atteignent jamais leur potentiel. Tu vois de la qualite sacrifiee sur l'autel des deadlines. Tu vois de l'excellence diluee dans l'urgence permanente.

Ce n'est pas une fatalite. C'est un pattern. Et comme tout pattern, il a des causes - des causes profondes que la superficie des methodologies n'a jamais vraiment adressees.

Les fractures actuelles dans la réalisation logicielle

La premiere fracture est la fracture entre la vision et l'execution. Dans trop d'organisations, la vision du produit vit dans la tete d'une poignee de personnes - souvent des product owners, des directeurs de produit, des founders - et s'etiole en traversant les couches de l'organisation jusqu'aux equipes de developpement. Ce que les developpeurs codent n'est souvent qu'une lointaine echo de la vision originale, traduite, retraduite, interpretee, simplifiee et parfois tout simplement mal comprise.

La Totalisation nomme cette fracture pour ce qu'elle est : une rupture du continuum. La vision, l'architecture, la realisation, le deploiement et l'evolution d'un logiciel devraient former un fil continu et coherent. Quand ce fil se rompt - et il se rompt constamment dans les organisations actuelles -, l'excellence devient impossible, non pas parce que les individus manquent de competence, mais parce que le systeme dans lequel ils travaillent brise systematiquement la coherence necessaire a l'excellence.

La deuxieme fracture est la fracture temporelle. Le monde du logiciel vit dans une presentite quasi absolue. Ce qui compte, c'est la prochaine release. Le prochain sprint. Le prochain meeting de statut. L'avant - la comprehension profonde du probleme, la conception reflechie, l'apprentissage patient - est systematiquement sacrifie au nom de la rapidite. L'apres - la maintenance, l'evolution, l'observabilite, la degradation gracieuse, la vie reelle du logiciel dans les mains des utilisateurs - est traite comme une preoccupation secondaire.

Cette fracture temporelle produit ce qu'on appelle communement la dette technique, mais qui est en realite beaucoup plus qu'une dette technique. C'est

une dette de vision. C'est le prix que l'on paie quand on refuse de voir le logiciel dans la totalite de son existence temporelle.

Les trois grandes fractures du logiciel

Fracture 1 - Vision versus execution : La vision s'etiole en traversant l'organisation.

Fracture 2 - Le passé et le futur sacrifiés au present : L'avant et l'apres sont ignorés au profit de l'urgence immediate.

Fracture 3 - L'humain etranger au systeme : Le logiciel est cree sans vraiment comprendre les humains qui vont le vivre.

La troisieme fracture est peut-etre la plus fondamentale : la fracture entre le logiciel et l'humain. On parle beaucoup d'experience utilisateur, de centricite client, de design thinking. Mais dans les faits, le logiciel est encore trop souvent cree par des techniciens pour des abstractions appelees « utilisateurs » plutot que par des gens qui comprennent profondement d'autres gens. L'empathie - reelle, profonde, incarnee - est rare dans le processus de creation logicielle. Et sans empathie, il ne peut pas y avoir d'excellence, parce que l'excellence logicielle est fondamentalement un service a l'humain.

Pourquoi la qualité est souvent sacrifiée

La question de la qualite est l'une des plus mal comprises dans le monde du logiciel. On croit souvent que la qualite est sacrifiee parce que les deadlines sont trop serrees, parce que les budgets sont trop contraints, parce que les equipes sont trop petites. Ces facteurs jouent un role, certes. Mais la cause profonde est ailleurs.

La qualite est sacrifiee parce qu'elle est percue comme un cout, et non comme un investissement. Elle est sacrifiee parce qu'on ne voit pas ce qu'on sacrifie - les

consequences d'une qualite insuffisante sont souvent differees, invisibles dans l'immediat, diluees dans le temps. Un bug qu'on n'a pas corrige aujourd'hui sera peut-etre un incident critique dans six mois. Une architecture qu'on a choisie pour sa simplicite immediate sera peut-etre un mur impossible a franchir dans deux ans.

La Totalisation propose une autre maniere de voir : la qualite n'est pas quelque chose qu'on ajoute apres. Elle n'est pas une phase du processus. Elle est une propriete du geste. Chaque geste de creation - chaque ligne de code, chaque decision d'architecture, chaque test - est soit un geste de qualite, soit un geste qui compromet la qualite. Il n'y a pas de zone neutre. Et cette conscience, cette responsabilite permanente envers la qualite, est au coeur de ce que la philosophie ORIGINALIUM appelle l'excellence totale.

Le paradoxe de la vitesse contre la qualité

On t'a sans doute dit, a un moment ou un autre de ta carriere, que tu devais choisir entre aller vite et aller bien. C'est l'un des mensonges les plus repandus - et les plus couteux - du monde du logiciel. Ce paradoxe apparent est en realite une illusion produite par une vision a court terme.

Voici ce que les donnees de decennies de projets logiciels montrent reellement : les equipes qui investissent dans la qualite des le depart livrent plus vite sur le long terme. Elles passent moins de temps a corriger des bugs. Elles passent moins de temps a essayer de comprendre un code qu'elles ont elles-memes ecrit quelques mois plus tot. Elles deploient avec plus de confiance. Elles repondent aux changements avec plus d'agilite - la vraie agilite, pas l'agilite de surface.

La vitesse apparente que l'on gagne en sacrifiant la qualite est une dette. Et comme toute dette, elle porte des interets. Des interets qui augmentent avec le temps. Des interets qui finissent par consommer une proportion croissante de la

capacite de l'equipe, jusqu'a ce que l'equipe ne soit plus capable de livrer quoi que ce soit de nouveau - elle est completement absorbee par le service de sa dette.

> *« Tu ne peux pas aller vraiment vite sur une route que tu as toi-meme trouee. La qualite n'est pas l'ennemi de la vitesse - c'est la condition de la vitesse durable. »*
>
> - Léo Michel Bélanger, ORIGINALIUM

Les symptômes du manque d'excellence

Le manque d'excellence dans le logiciel se manifeste rarement de maniere spectaculaire et soudaine. Il s'installe progressivement, insidieusement, comme une erosion. Les symptomes sont souvent mal interpretes - on les attribue a de mauvaises personnes, a de mauvais outils, a de mauvais processus - alors qu'ils sont les signes d'une absence de philosophie fondatrice.

Il y a d'abord l'entropie acceleree. Un logiciel sans vision totale vieillit mal. Chaque nouvelle fonctionnalite ajoute de la complexite sans vraiment l'integrer dans un tout coherent. Le code devient de plus en plus difficile a comprendre, a modifier, a tester. Les equipes passent de plus en plus de temps a gerer la complexite existante et de moins en moins de temps a creer de la valeur nouvelle. C'est le syndrome de la « dette technique » dans sa forme la plus avancee.

Il y a ensuite la fragmentation du savoir. Sans une vision totale partagee, le savoir se fragmente. Chaque developpeur connait son bout de code et comprend mal le reste. Chaque equipe connait son domaine et se desinteresse des autres. Le logiciel devient une collection de territoires plutot qu'un espace integre. Et quand quelqu'un part - quand ce developpeur qui connaissait « ce module-la » quitte l'organisation -, c'est un trou qui apparait, un trou que personne ne peut vraiment combler parce que la connaissance n'etait pas partagee, n'etait pas documentee, n'etait pas totalise.

Il y a enfin l'epuisement du sens. Les equipes qui travaillent dans un logiciel sans vision totale finissent par perdre le sens de ce qu'elles font. Elles executent des taches. Elles completent des stories. Elles font passer des builds. Mais elles ne vivent plus l'experience de creer quelque chose d'excellent, de contribuer a quelque chose qui les depasse. C'est cet epuisement du sens - et non les salaires ou les avantages sociaux - qui est souvent la vraie cause du roulement de personnel dans le monde du logiciel.

Le tableau BEASTEST offre un miroir pour voir ou en est ton organisation dans cette progression. Ses sept niveaux de maturite - du **ROUGE** au **VIOLET**, en passant par l'**ORANGE**, le **JAUNE**, le **VERT**, le **BLEU** et l'**INDIGO** - ne sont pas des jugements. Ce sont des etapes sur un chemin d'excellence continu, des reperes pour savoir ou tu en es et vers ou tu te diriges. La Totalisation est le fil qui traverse tous ces niveaux et les unit dans une vision coherente du voyage vers l'excellence.

Niveau BEASTEST	Couleur	Caractéristique dominante dans le contexte de la fracture
Niveau 1	**ROUGE**	Chaos et réactivité totale. Absence de vision. Qualité inexistante.
Niveau 2	**ORANGE**	Conscience émergente des problèmes. Premières tentatives de structure.
Niveau 3	**JAUNE**	Pratiques définies mais appliquées de manière inconstante.
Niveau 4	**VERT**	Cohérence dans l'application des pratiques. Vision partagée émergente.

Niveau BEASTEST	Couleur	Caractéristique dominante dans le contexte de la fracture
Niveau 5	**BLEU**	Optimisation continue. La qualité est une propriété du geste, pas une phase.
Niveau 6	**INDIGO**	Prédictibilité et excellence systémique. La Totalisation est incarnée.
Niveau 7	**VIOLET**	Innovation et transcendance. Le logiciel comme art total. L'excellence comme état d'être.

CHAPITRE 3

L'art versus la technique dans le logiciel

Ce qui distingue une grande oeuvre logicielle d'un logiciel fonctionnel mais sans ame, ce n'est pas la technologie utilisee. Ce n'est pas non plus la methodologie suivie. C'est quelque chose de bien plus fondamental : la presence ou l'absence d'un artisan qui a incarne une vision totale dans chacun de ses gestes.

Q

uand Linus Torvalds a commence a ecrire le noyau Linux dans sa chambre d'etudiant a Helsinki en 1991, ce n'etait pas un technicien en train d'executer des specifications. C'etait un artisan en train de donner forme a une vision. Quand les premiers developpeurs de NeXT ont cree le systeme qui est devenu la fondation de macOS, ils n'etaient pas en train de remplir des story points. Ils etaient en train

de sculpter quelque chose qu'ils voulaient beau, elegant, et durable. L'art dans le logiciel ne s'oppose pas a la technique - il la transcende, il la fait passer d'un niveau de simple execution a un niveau de creation veritable.

Ce qui distingue l'artisan du technicien en génie logiciel

La distinction entre l'artisan et le technicien n'est pas une question de competence. Un technicien peut etre extremement competent - maitriser plusieurs langages, connaitre les design patterns, comprendre les architectures distribuees. Un artisan peut ne maitriser qu'un seul domaine. La difference n'est pas dans la largeur des competences. Elle est dans la nature de la relation qu'ils entretiennent avec leur travail.

Le technicien applique. Il prend un probleme, il identifie la technique appropriee, il l'applique, il passe au suivant. Son rapport au travail est transactionnel : il echange sa competence technique contre un resultat specifie. Il y a de la rigueur dans cette approche, et de la valeur. Mais il y a une limite : la technique, appliquee sans vision, produit des solutions correctes mais pas necessairement excellentes.

L'artisan, lui, s'interroge. Il se demande non seulement *comment* faire, mais *pourquoi* faire ainsi, et *si* c'est vraiment ce qu'il faut faire. Il porte en lui une vision de ce que le logiciel devrait etre - pas seulement ce qu'il devrait faire - et chaque decision technique est filtrée par cette vision. Un artisan du logiciel qui ecrit une fonction n'est pas seulement en train de produire quelque chose qui fonctionne. Il est en train de contribuer a quelque chose qui doit aussi etre lisible, maintenable, coherent avec l'ensemble, et potentiellement comprensible par quelqu'un qui lira ce code dans cinq ans et aura besoin de le modifier en quelques heures.

Portrait de l'artisan du logiciel

L'artisan du logiciel se reconnaît a cinq traits fondamentaux :

1. L'intention totale : Chaque geste est conscient et oriente par une vision.

2. La fierté du detail : Les parties invisibles sont soignees autant que les parties visibles.

3. La responsabilité etendue : Il se sent responsable non seulement de ce qu'il livre, mais de ce que son travail devient dans le temps.

4. L'humilite apprenante : Il sait que son art est toujours en devenir, toujours perfectible.

5. La vision totale : Il ne perd jamais de vue l'ensemble, meme quand il travaille sur le detail le plus infime.

La dimension humaine de la qualité

Il y a dans la qualite logicielle une dimension profondement humaine que les frameworks de mesure de la qualite peinent a capturer. La qualite, dans sa nature la plus profonde, n'est pas une propriete du code. C'est une propriete de la relation entre le code et les humains qui le cree et qui l'utilise.

Pense a un moment ou tu as ouvert un fichier de code - le tien ou celui de quelqu'un d'autre - et ou tu as ressenti immediatement, visceralement, que c'etait bien fait. Pas parce que tu avais fait tourner des metriques de qualite. Pas parce qu'un outil d'analyse statique t'avait donne un score. Mais parce que quelque chose dans la maniere dont le code etait structure, dans le choix des noms de variables, dans la facon dont les abstractions etaient posees, te disait que la personne qui avait ecrit ca avait pense a toi - a la personne qui allait lire ce code apres eux.

C'est ca, la dimension humaine de la qualite. C'est l'empathie encode dans le code. C'est la consideration pour le futur - pour le prochain developpeur, pour le prochain utilisateur, pour la prochaine version du systeme - qui traverse chaque

decision de l'artisan. Et cette forme d'empathie, cette attention a l'autre dans l'acte de creation, est au coeur de ce que la philosophie ORIGINALIUM entend par « realiser totalement ».

La Totalisation te demande d'etendre ta conscience dans le temps et dans l'espace. Pas seulement de voir le code que tu es en train d'ecrire, mais de voir le developpeur qui va le lire dans deux ans. Pas seulement de voir la fonctionnalite que tu es en train de livrer, mais de voir l'utilisateur qui va tenter de l'utiliser a minuit sur un reseau mobile lent. Pas seulement de voir la solution que tu proposes, mais de voir le probleme que tu essaies vraiment de resoudre, sous les symptomes presents.

L'intuition cultivée par l'expérience

L'une des resistances les plus courantes a l'idee que le logiciel est un art vient de la conviction que l'art est subjectif et donc incontrôlable, alors que le logiciel devrait etre objectif et mesurable. Cette opposition est une fausse dichotomie.

Dans toutes les grandes traditions d'excellence humaine - que ce soit la cuisine, l'architecture, la musique, la medecine -, il y a une forme de connaissance que les anciens Grecs appelaient *techne* et qui correspond exactement a ce que les grands praticiens du logiciel developpent avec les annees : une intuition cultivee par l'experience, une capacite a percevoir des patterns et des opportunites que la methode seule ne peut pas identifier.

Un grand chirurgien ne suit pas seulement le protocole. Il s'adapte. Il voit des choses que le protocole ne peut pas anticiper. Son experience lui a donne une sorte de vision peripherique - une capacite a tenir simultanement la procedure et le patient, la technique et la situation unique qui se presente devant lui. C'est cette meme capacite que developpe un grand artisan du logiciel : la capacite a tenir simultanement la solution immediate et le systeme global, la fonctionnalite demandee et les implications a long terme de chaque choix technique.

Cette intuition ne s'enseigne pas directement. Elle se cultive, lentement, par la pratique reflexive, par l'attention consciente portee a chaque experience, par la volonte de comprendre non seulement ce qui a fonctionne mais pourquoi ca a fonctionne, et ce qui n'a pas fonctionne et pourquoi ca n'a pas fonctionne. C'est un chemin long, exigeant, et profondement satisfaisant - exactement comme tout chemin vers une veritable maitrise.

> *« L'intuition de l'artisan n'est pas l'oppose de la methode du scientifique. C'est la methode du scientifique qui a tellement ete integree qu'elle est devenue naturelle, fluide, immediate - une seconde nature plutot qu'une premiere contrainte. »*
>
> - Léo Michel Bélanger, ORIGINALIUM

Totalisation comme synthèse de l'art et de la rigueur

La philosophie ORIGINALIUM ne choisit pas entre l'art et la rigueur. Elle les reconcilie dans le concept de Totalisation. Cette reconciliation est au coeur de ce que le cadre BEASTEST cherche a incarner dans ses sept niveaux.

Au niveau **ROUGE**, on n'a ni art ni rigueur - on a du chaos. On fait ce qu'on peut, quand on peut, comme on peut. La survie domine la creation. Au niveau **ORANGE**, une premiere forme de rigueur emerge - des processus se mettent en place, des pratiques se definissent. Mais l'art est encore absent - la rigueur est mecanique, appliquee sans comprehension profonde.

C'est dans les niveaux intermediaires - **JAUNE**, **VERT**, **BLEU** - que la tension entre art et rigueur se joue le plus intensement. C'est la que les equipes doivent apprendre a integrer les deux, a ne pas laisser la rigueur etouffer la creativite ni

laisser la creativite compromettre la fiabilite. C'est un equilibre difficile, et il n'est jamais definitif - il doit etre constamment reconqueri.

Au niveau **INDIGO** et au niveau **VIOLET**, quelque chose de remarquable se produit : l'art et la rigueur cessent d'etre perçus comme deux forces distinctes. Ils deviennent une seule chose. La rigueur est devenue si naturelle qu'elle se manifeste dans chaque geste de creation sans effort conscient - elle est integree, incarnee, totalisee. Et l'art peut s'exprimer pleinement, libere de la contrainte de « faire attention », parce que la qualite est desormais une propriete du geste et non le resultat d'un effort supplementaire.

C'est cet etat - ou la rigueur et la creativite sont reconciliees dans une excellence fluide et naturelle - que la Totalisation cherche a realiser. Et c'est pourquoi ce livre s'appelle « L'origine de l'art de l'excellence » : parce que l'excellence veritable dans le logiciel est un art, et que cet art a une origine philosophique profonde que ce manifeste cherche a eclairer.

CHAPITRE 4

Les origines philosophiques de l'excellence

L'excellence n'est pas une invention du genie logiciel. Elle est une aspiration aussi ancienne que la civilisation humaine elle-meme. Comprendre ses origines philosophiques, c'est comprendre pourquoi elle est si rare, pourquoi elle est si precieuse, et comment la philosophie de la Totalisation la reintroduit dans le contexte contemporain de la creation logicielle.

L

es Grecs de l'Antiquite avaient un mot pour ce que nous cherchons : *arete*. Ce mot est habituellement traduit par « excellence » ou « vertu », mais ces traductions

sont insuffisantes. L'*arete* grecque designe quelque chose de plus precis et de plus exigeant : la pleine realisation de ce qu'une chose ou un etre est capable d'etre. L'*arete* d'un couteau, c'est d'etre parfaitement aiguise. L'*arete* d'un cheval, c'est d'etre rapide, fort et endurant. L'*arete* d'un homme, disait Aristote, c'est d'exercer pleinement sa raison au service du bien.

Cette conception est profondement pertinente pour le logiciel. L'*arete* d'un logiciel, c'est de realiser pleinement ce qu'il est capable d'etre - non pas seulement de faire ce qu'il est cense faire, mais de le faire de maniere tellement complete, tellement coherente, tellement integre dans le contexte de ceux qui l'utilisent, que son excellence est palpable, immediate, indeniable.

D'ou vient l'idée d'excellence dans la civilisation humaine

L'idee d'excellence traverse toutes les grandes civilisations humaines, meme si elle prend des formes differentes selon les cultures et les epoques. Dans la tradition japonaise, on parle de *shokunin kishitsu* - « l'ame de l'artisan » - une devotion totale a la maitrise de son metier qui transcende la simple production de biens pour atteindre une forme d'expression de soi et de contribution au monde. Le cuisinier de sushi qui passe dix ans a apprendre uniquement a preparer le riz avant de toucher au poisson n'est pas en train de perdre son temps. Il est en train d'incerner une philosophie de la totalisation : la maitrise totale vient d'une attention totale portee a chaque composant, aussi simple qu'il soit en apparence.

Dans la tradition des batisseurs de cathedrales medievales, on retrouve une autre expression de cette philosophie. Ces batisseurs travaillaient sur des oeuvres qu'ils savaient ne jamais voir achevees. La cathedrale de Chartres a pris plus d'un siecle a construire. Personne parmi ceux qui ont pose les premieres pierres n'a vu la flèche s'elever. Et pourtant, ils ont travaille avec la meme precision, le meme soin, la meme attention au detail que s'ils allaient voir l'oeuvre terminee le lendemain.

Parce que pour eux, l'excellence n'etait pas une finalite - c'etait une maniere d'etre dans chaque geste de creation.

C'est exactement cette philosophie que la Totalisation cherche a reintroduire dans le logiciel. Le logiciel que tu crées aujourd'hui sera peut-etre utilise - et modifie, et evolue - dans dix ans, par des personnes que tu ne connais pas, dans des contextes que tu n'imagines pas. L'excellence que tu inscris dans chaque decision, dans chaque ligne de code, dans chaque architecture que tu dessines, est une forme de transmission. C'est un cadeau que tu fais a l'avenir.

L'excellence appliquée au logiciel : une vision ORIGINALIUM

La vision ORIGINALIUM de l'excellence logicielle repose sur quatre convictions fondamentales qui s'articulent autour du concept de Totalisation.

La premiere conviction est que **l'excellence est une propriete emergente**. Elle ne s'obtient pas en faisant tres bien une seule chose. Elle emerge de la coherence entre de nombreuses dimensions : la qualite du code, la pertinence de l'architecture, la profondeur de la comprehension du probleme, la qualite de la communication dans l'equipe, la clarte de la vision produit, la solidite des pratiques de test, l'attention portee aux operations et a la maintenabilite. L'excellence totale emerge quand toutes ces dimensions sont elevees simultanement et maintenues en coherence - quand elles sont totalisees.

La deuxieme conviction est que **l'excellence est un chemin, pas une destination**. Il n'y a pas de moment ou un logiciel est « excellent » de maniere definitive. L'excellence est un etat dynamique, qui doit etre constamment maintenu, constamment reconquis a travers l'evolution du systeme, des technologies, des besoins des utilisateurs et du contexte organisationnel. C'est pourquoi la philosophie ORIGINALIUM insiste sur le « en continu, sans interruption

» - parce que l'excellence qui s'interrompt n'est pas de l'excellence. C'est de la qualite ponctuelle, qui va inevitablement se degrader.

La troisieme conviction est que **l'excellence est collective**. Aucun individu, aussi brillant soit-il, ne peut produire a lui seul un logiciel excellent dans le contexte de la complexite moderne. L'excellence dans le logiciel contemporain est necessairement une oeuvre collective - elle emerge de la qualite des interactions entre les personnes, de la vision partagee, de la confiance mutuelle, de la capacite collective a integrer des perspectives diverses dans un tout coherent. La Totalisation est donc aussi une philosophie du collectif, une maniere de voir l'equipe comme un organisme de creation plutot qu'une agregation de ressources.

La quatrieme conviction est peut-etre la plus difficile a accepter dans le contexte actuel : **l'excellence prend du temps**. Pas indefiniment. Mais plus que ce qu'on lui accorde generalement. La profondeur - la vraie profondeur de comprehension d'un probleme, la vraie qualite d'une architecture, la vraie solidite d'un systeme - ne peut pas etre rush. Elle ne peut pas etre comprimee dans un sprint de deux semaines quand le probleme demande un mois de reflexion. Et reconnaitre cela, defendre cela dans un monde qui valorise la vitesse avant tout, est lui-meme un acte de courage - un acte de fidelite a l'excellence.

> *« L'excellence n'est jamais le fruit de la precipitation. C'est le fruit de la patience - non pas la patience passive de celui qui attend, mais la patience active de celui qui sait que certaines choses demandent le temps qu'elles demandent, et qui ne marchandera pas sur ce temps. »*
>
> - Léo Michel Bélanger, ORIGINALIUM

Le continuum avant-pendant-apres comme principe fondateur

L'une des contributions les plus distinctives de la philosophie ORIGINALIUM est l'articulation du principe du continuum avant-pendant-apres comme fondement de l'excellence logicielle. Ce principe est si simple a enoncer qu'il semble presque banal - et pourtant, son application rigoureuse et complete dans la pratique quotidienne du developpement logiciel est transformatrice.

L'**avant** designe tout ce qui precede l'acte de creation : la comprehension du probleme, la definition de la vision, la conception de l'architecture, la planification de la qualite, la preparation des environnements, la formation des equipes, l'etablissement des principes directeurs. Dans la pratique actuelle, l'avant est systematiquement raccourci. On commence a coder trop vite. On deploie des architectures avant de les avoir suffisamment pensees. On recrute des equipes sans les former a une vision commune. Le resultat est previsible : on passe l'essentiel du pendant a reparer ce qu'on aurait du resoudre dans l'avant.

Le **pendant** designe l'acte de creation lui-meme : le codage, les revues de code, les tests, les deploiements, les iterations, les ajustements. C'est la phase que le monde du logiciel connait le mieux - c'est celle sur laquelle l'essentiel des methodologies se concentrent. Mais meme dans le pendant, la Totalisation apporte une contribution essentielle : elle insiste sur la coherence continue entre chaque geste de creation et la vision totale du systeme. Chaque pull request, chaque session de pair programming, chaque discussion d'architecture est une opportunite de totaliser - d'integrer ce qu'on fait maintenant dans la vision de ce qu'on construit globalement.

L'**apres** est peut-etre la dimension la plus negligee dans le genie logiciel contemporain. Une fois qu'un systeme est en production, l'attention se detourne rapidement vers le prochain projet. La maintenance est vue comme un fardeau plutot qu'une composante naturelle de l'excellence. L'observabilite - la capacite a

voir ce qui se passe dans le systeme en production - est souvent pensee apres coup, quand un incident l'impose plutot que par conception. Et la fin de vie d'un systeme - le moment ou il doit etre remplace ou retire - est rarement pensee a l'avance, ce qui produit des situations de migration catastrophiques.

La Totalisation change fondamentalement la maniere de penser l'apres. Elle dit : l'excellence d'un systeme inclut necessairement la qualite de son vieillissement, la dignite de son evolution, la fluidite de sa transition vers ce qui le remplacera. Un logiciel excellent ne s'effondre pas dans l'age - il evolue gracieusement. Et cette capacite d'evolution gracieuse se concoit dans l'avant, s'inscrit dans le pendant, et se realise dans l'apres.

La totalisation comme acte quotidien

Il serait facile de lire tout ce qui precede et de voir la Totalisation comme un ideal lointain, une aspiration philosophique belle mais difficilement accessible dans la realite quotidienne du developpement logiciel, avec ses contraintes, ses pressions, ses urgences et ses compromis inevitables. Cette lecture serait une erreur profonde.

La Totalisation n'est pas un ideal de perfection. C'est une orientation. C'est une direction permanente dans laquelle on choisit de se deplacer, sachant que la destination n'est jamais definitivement atteinte, mais que l'orientation elle-meme transforme chaque geste, chaque decision, chaque interaction.

La Totalisation comme acte quotidien, ca ressemble a ceci : avant de commencer a coder une fonctionnalite, prendre le temps de vraiment comprendre le probleme qu'elle est censee resoudre - pas le probleme tel qu'il est exprime dans la user story, mais le probleme profond que vit l'utilisateur. Ca ressemble a ceci : quand on ecrit un test, penser non seulement a la validation immediate mais a ce que ce test dit aux developpeurs futurs sur l'intention du code qu'il teste. Ca ressemble a ceci : quand on prend une decision d'architecture, se demander non seulement si

elle fonctionne maintenant, mais si elle sera encore comprehensible et maintenable dans trois ans.

La Totalisation comme acte quotidien, c'est aussi la capacite d'arreter. D'arreter une discussion qui tourne en rond pour la rencadrer dans la vision globale. D'arreter un sprint qui va dans la mauvaise direction pour recalibrer plutot que de continuer et de livrer quelque chose d'inadequat. D'arreter une architecture qui montre ses limites pour la reconcevoir plutot que de continuer a la patcher jusqu'a ce qu'elle s'effondre.

Ces moments d'arret - ces moments de recalibration - sont eux-memes des actes de Totalisation. Ils sont l'expression de la conviction que l'excellence totale vaut mieux que la livraison partielle. Et ils sont, paradoxalement, souvent ce qui permet d'aller le plus vite sur le long terme.

La Totalisation en trois gestes quotidiens

Geste 1 - Ancrer : Au debut de chaque session de travail, ancre-toi dans la vision totale. Pas dans la tache immediate - dans la vision. Ou est-ce que cette tache s'inscrit dans l'ensemble ? Quelle est son importance relative ? Quelles sont ses implications au-dela de l'immediat ?

Geste 2 - Integrer : Au cours du travail, integre continuellement ce que tu fais dans le tout. Chaque decision que tu prends, demande-toi : est-ce que ca s'integre bien dans l'ensemble ? Est-ce que ca cree de la coherence ou de la fragmentation ?

Geste 3 - Transmettre : A la fin de chaque session, assure-toi que ce que tu as fait peut etre compris, maintenu et evolue par quelqu'un d'autre. La transmission est l'acte final de la Totalisation - c'est ce qui fait que ton travail continue de vivre au-dela de ton implication directe.

Ces trois gestes - ancrer, integrer, transmettre - sont la Totalisation rendue concrete. Ils ne necessitent pas de methode supplementaire, pas de framework additionnel, pas d'outil specifique. Ils necessitent une intention. Une attention. Et une fidelite a la vision de l'excellence totale qui ne se compromet pas dans l'urgence du quotidien.

C'est la un des paradoxes de la Totalisation : elle est en meme temps une philosophie grandiose et une pratique humble. Elle parle d'art et d'excellence et de vision totale, et en meme temps, elle se realise dans des gestes minuscules et quotidiens. C'est exactement comme la meditation : les principes qui la fondent peuvent sembler eleves, mais la pratique consiste simplement a s'asseoir, a respirer, et a revenir encore et encore a l'objet de son attention. La Totalisation, c'est revenir encore et encore a la vision de l'ensemble, dans chaque geste de creation logicielle.

> *« L'excellence totale n'est pas un etat qu'on atteint une fois pour toutes. C'est une facon d'etre presente dans chaque geste, chaque decision, chaque conversation. C'est un engagement renouvelable a chaque instant - un oui silencieux dit a la totalite de ce qu'on fait. »*
>
> - Léo Michel Bélanger, ORIGINALIUM

Note de fin de Partie A - Apercu de la Partie B

Tu viens de traverser l'Origine. Tu as rencontre la Totalisation dans sa double dimension - linguistique et philosophique. Tu as contemple l'etat actuel du monde du logiciel avec l'honnetet d'un diagnostic courageux. Tu as explore la distinction entre l'art et la technique dans la creation

logicielle. Et tu as plonge dans les origines philosophiques profondes de l'excellence, depuis l'*arete* grecque jusqu'au continuum avant-pendant-apres de la philosophie ORIGINALIUM.

Mais une origine ne suffit pas. Une vision ne se contemple pas - elle se deploie. La Partie B de ce livre - dont le titre provisoire est **« ŢŢĻŞŢ~ - L'architecture de l'excellence : le cadre BEASTEST »** - prendra la vision que tu as decouverte ici et commencera a l'articuler dans sa structure complete.

Dans la Partie B, tu plongeras dans le coeur du cadre BEASTEST et ses sept niveaux de maturite coloree. Tu comprendras non seulement ce que chaque niveau signifie dans l'abstrait, mais ce qu'il implique concretement dans la maniere dont une equipe pense, travaille, communique et cree. Tu verras comment le passage du **ROUGE** au **VIOLET** n'est pas une ascension lineaire mais une transformation qualitative - un changement profond dans la nature meme de la relation qu'une equipe entretient avec son travail.

Tu decouvriras egalement comment la Totalisation s'incarne dans chacun des sept niveaux - comment elle est presente des le debut du chemin, meme dans sa forme la plus embryonnaire au niveau **ROUGE**, et comment elle s'epanouit pleinement dans sa forme la plus accomplie au niveau **VIOLET**.

La Partie A t'a donne les fondations. La Partie B te donnera la structure. Et la Partie C - « ŢŢĻŞŢ~ - La vie de l'excellence : incarner la Totalisation au quotidien » - te donnera la chair et le souffle de cette philosophie dans la realite concrete de la creation logicielle. Mais avant tout cela, il fallait l'Origine. Et tu l'as maintenant.

La Totalisation n'est pas une methode qu'on apprend.

C'est une vision qu'on incarne.

Et toute incarnation commence par une origine.

ŢŢĻŞŢ~

ŢŢĻŞŢ~ - L'origine de l'art de l'excellence pour la realisation dans le domaine du logiciel - Partie A

© Léo Michel Bélanger - Tous droits réservés | Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM | ŢŢĻŞŢ~ | Montreal, Quebec, Canada

Aucune partie de ce document ne peut etre reproduite, transmise ou utilisee sous quelque forme que ce soit sans l'autorisation ecrite explicite de l'auteur.